Memory Issues



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



Usual Memory Issues

- Programming (almost) always involves accessing memory
- If done incorrectly, a large variety of errors can be triggered
  - Segmentation Faults can happen when accessing invalid memory addresses (NULL pointers or use-after-free for instance)
  - Buffer Overflows can happen if accessing a buffer outside its boundaries
  - Memory Leaks when allocating memory and forgetting to free it after usage
- Fortunately, there are tools to debug these errors



Segmentation Faults

- Segmentation Faults are generated by the kernel when a program tries to access a memory area that it is not allowed to or to access it in an incorrect way
  - Might be generated by a write on a read only memory zone
  - Can also be triggered when trying to execute memory that is not executable

```
int *ptr = NULL;
*ptr = 1;
```

- Execution will yield a `Segmentation fault` message in the terminal

```
$ ./program
Segmentation fault
```



Buffer Overflows

- Buffer Overflows are easily triggered when accessing an array outside of its boundaries (most often past the end)
- Such access might generate a crash or not depending on the access
  - Writing past the end of a `malloc()`'ed array will most often overwrite the malloc data structure leading to corruption
  - Writing past the end of an array allocated on the stack can corrupt data on the stack
  - Reading past the end of an array might generate a segfault but not always, this depends on the area of memory that is accessed

```
uint32_t *array = malloc(10 * sizeof(*array));
array[10] = 0xDEADBEEF;
```



Memory Leaks

- Memory leaks are another class of memory errors that will not directly trigger a crash but will exhaust the system memory (sooner or later)
- This happens when allocating memory in your program and not releasing it after using it
- Can trigger in production when the program runs for a very long time
  - Better to debug that kind of problem early in the development process

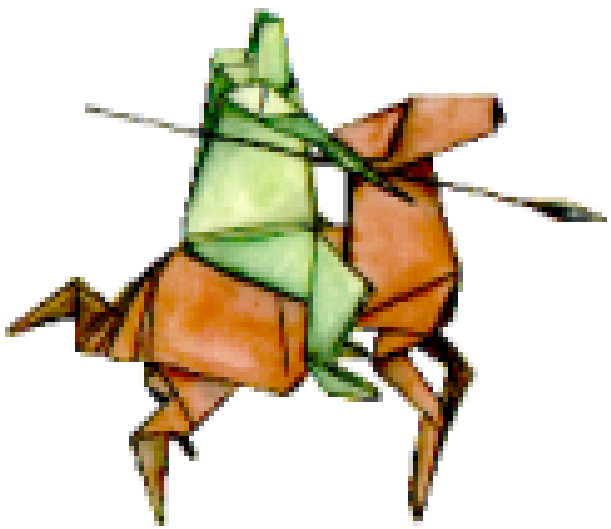
```
void func1(void) {
  uint32_t *array = malloc(10 * sizeof(*array));
  do_something_with_array(array);
}
```



Valgrind memcheck

Valgrind (1/2)

- Valgrind is an instrumentation framework for building dynamic analysis tools
- `valgrind` is also a tool that is based on this framework and provides a memory error detector, heap profilers and others profilers.
- It is supported on all the popular platforms: Linux on x86, x86_64, arm (armv7 only), arm64, mips32, s390, ppc32 and ppc64.



Valgrind (2/2)

- Works by adding its own instrumentation to your code and then running in on its own virtual cpu core. Significantly slows down execution, and thus is suited for debugging and profiling
- *Memcheck* is the default *valgrind* tool and it detects memory-management errors
  - Access to invalid memory zones, use of uninitialized values, memory leaks, bad freeing of heap blocks, etc
  - Can be run on any application, no need to recompile them

```
$ valgrind --tool=memcheck --leak-check=full <program>
```





Valgrind Memcheck usage and report

```
$ valgrind ./mem_leak
==202104== Memcheck, a memory error detector
==202104== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==202104== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==202104== Command: ./mem_leak
==202104==
==202104== Conditional jump or move depends on uninitialised value(s)
==202104==    at 0x109161: do_actual_jump (in /home/user/mem_leak)
==202104==    by 0x109187: compute_address (in /home/user/mem_leak)
==202104==    by 0x1091A2: do_jump (in /home/user/mem_leak)
==202104==    by 0x1091D7: main (in /home/user/mem_leak)
==202104==
==202104== HEAP SUMMARY:
==202104==     in use at exit: 120 bytes in 1 blocks
==202104==   total heap usage: 1 allocs, 0 frees, 120 bytes allocated
==202104==
==202104== LEAK SUMMARY:
==202104==    definitely lost: 120 bytes in 1 blocks
==202104==    indirectly lost: 0 bytes in 0 blocks
==202104==      possibly lost: 0 bytes in 0 blocks
==202104==    still reachable: 0 bytes in 0 blocks
==202104==         suppressed: 0 bytes in 0 blocks
==202104== Rerun with --leak-check=full to see details of leaked memory
```



Valgrind and VGDB

- Valgrind can also act as a GDB server which can receive and process commands. One can interact with valgrind gdb server either with a gdb client, or directly with `vgdb` program (provided with valgrind). `vgdb` can be used in different ways:
  - As a standalone CLI program to send "monitor" commands to valgrind
  - As a relay between a gdb client and an existing valgrind session
  - As a server to drive multiple valgrind sessions from a remote gdb client
- See man 1 vgdb for available modes, commands and options



Using GDB with Memcheck

- *valgrind* allows to attach with GDB to the process that is currently analyzed.

```
$ valgrind --tool=memcheck  --leak-check=full --vgdb=yes --vgdb-error=0 ./mem_leak
```

- Then attach gdb to the valgrind gdbserver using *vgdb*

```
$ gdb ./mem_leak
(gdb) target remote | vgdb
```

- If valgrind detects an error, it will stop the execution and break into GDB.

```
(gdb) continue
Continuing.

Program received signal SIGTRAP, Trace/breakpoint trap.
0x0000000000109161 in do_actual_jump (p=0x4a52040) at mem_leak.c:5
5   if (p[1])
(gdb) bt
#0  0x0000000000109161 in do_actual_jump (p=0x4a52040) at mem_leak.c:5
#1  0x0000000000109188 in compute_address (p=0x4a52040) at mem_leak.c:11
#2  0x00000000001091a3 in do_jump (p=0x4a52040) at mem_leak.c:16
#3  0x00000000001091d8 in main () at mem_leak.c:27
```



Electric Fence



libefence (1/2)

- *libefence* is more lightweight than *valgrind* but less precise
- Allows to catch two types of common memory errors
  - Buffer overflows and use after free
- *libefence* will actually trigger a segfault upon the first error encountered in order to generate a coredump.
- Uses a shared library that can either be linked with statically (`-lefence`) or preloaded using `LD_PRELOAD`.

```
$ gcc -g program.c -o program
$ LD_PRELOAD=libefence.so.0.0 ./program

  Electric Fence 2.2 Copyright (C) 1987-1999 Bruce Perens <bruce@perens.com>
Segmentation fault (core dumped)
```



libefence (2/2)

- Upon segfault, a coredump will be generated in the current directory
- This coredump can be opened with GDB and will pinpoint the exact location where the error happened

```
$ gdb ./program core-program-3485
Reading symbols from ./libefence...
[New LWP 57462]
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Core was generated by `./libefence'.
Program terminated with signal SIGSEGV, Segmentation fault.
#0  main () at libefence.c:8
8    data[99] = 1;
(gdb)
```

Practical lab - Debugging Memory Issues



Debug various memory issues using specific tooling

- Memory leak and misbehavior detection with *valgrind* and *vgdb*.